When the Interval Is Smaller Than the Instrument: Two Ways Streaming Latency Benchmarks Fail on Sub-Millisecond Paths

Gustavo Pedro Ricou

Abstract—**Streaming brokers are compared on sub-millisecond paths using instruments whose own timescales are larger than the intervals they report. One ratio governs what follows: a timescale belonging to the instrument, divided by the interval being measured. We report two consequences of that ratio exceeding one, both invisible in the output. First, a measured latency can come out negative, because one of the two stamps is written by a thread that must first be scheduled, and that stall can outlast the delivery being timed. A sign check rejected $1{,}321$ of $2{,}266$ runs. Across $738{,}730$ events the acknowledgement-referenced span inverts $62{,}264$ times while the send-referenced span, a causal chain, never does, on one clock by construction, so the cause is scheduling and not synchronisation. Real-time priority on the stamping threads, at unchanged utilisation, collapses the rate 7–$80\times$. Second, the most widely used cross-broker benchmark admits a sample only when a millisecond-quantised difference is positive and counts none of what it drops: over 71 embedded-mode runs reporting a median on the millisecond grid it computed that median from 0.36% to 100% of the samples it took, and the retained fraction follows the arithmetic of the send-interval-to-quantum ratio. The remedy is one line: dither the send instant, and publish the retention rate.**

Index Terms—**Performance measurement, measurement uncertainty, timing, synchronization, operating system scheduling, quantization, benchmark testing, message systems.**

I. Introduction

STATED plainly, this paper reports two ways a latency benchmark can be wrong about its own measurements, and one number that governs both.

A benchmark times a message by stamping a clock when the message is sent and again when it arrives, then subtracting. Both stamps are written by software that must first be given a core to run on. On a busy machine that wait can be longer than the interval being measured, and then the subtraction produces a negative delay: not because anything arrived before it was sent, but because the stamp meant to mark the start was written late. Separately, the most widely used benchmark for comparing brokers stamps in whole milliseconds and keeps only positive differences. On a path faster than a millisecond it therefore deletes samples, and how many it deletes is fixed by the arithmetic of the send rate rather than by chance. Neither failure appears in the output. Both are caught by checks that cost nothing: a sign test, and a count of what was thrown away.

The two failures share one number. Write T_{true} for the interval being measured and put beside it whichever timescale the instrument contributes, the scheduling stall before a stamp is written or the resolution of the stamp itself. When that instrument timescale exceeds T_{true}, the measurement stops describing the system and starts describing the instrument. Sub-millisecond broker paths are where that ratio crosses one for tooling in current use.

A. Contributions

1) **A deletion law for benchmark samples, established by manipulation** (Section IV). The OpenMessaging Benchmark admits a latency sample only when a millisecond-quantised difference is positive, with no counter for what it discards. Across the 71 instrumented runs that report a median of 1.0 or 2.0 ms it computed that median from 0.36% to 100% of the samples taken, and over the full ledger of 223 runs the retained fraction reaches as low as 0.0044%. Retention follows arithmetic the operator never sees: with send interval over quantum equal to p/q in lowest terms, a run retains one of the $q{+}1$ grid fractions bracketing T_{true}/τ. Fourteen arms across seven denominators behave as positioned, 9 of 10 powered arms reject a continuum null after correction, and a pre-registered payload manipulation moves an arm onto a grid vertex and off it.
2) **The inversion mechanism, established by manipulation on one clock** (Section V). An inversion occurs when the stall between an event and the clock read that labels it outlasts the interval being measured, so $\Pr[\text{inversion}] = \Pr[\text{stall} > T_{\text{true}}]$. Real-time priority on the stamping threads at fixed utilisation collapses the rate 7–$80\times$ across eight matched pairs; identical utilisation reached by two core geometries differs $2.07\times$; lengthening the true transport $77\times$ *lowers* the rate; an unfitted kernel trace predicts the observed rate to within a third. The span that inverts is referenced to an acknowledgement, and the send-referenced span over the same $738{,}730$ events never inverts, which locates the fault in the reference stamp rather than in the delivery.
3) **An audit, and the reporting rules that follow** (Sections III-D and VII-A). A sign check applied uniformly to all 2,266 runs rejected 1,321 of them, including every run behind our own first result. The practice is older than this paper: Paxson [1] treated physically impossible transit times as evidence of instrument failure and published the proportion of traces affected. What we add

G. P. Ricou is with the School of Computer Science and Statistics, Trinity College Dublin, Dublin 2, Ireland. E-mail: pedrorig@tcd.ie.

is a pre-registered run-level rule for a modern streaming benchmark, and the observation that the retention rate and the sign of each discard belong beside every reported latency.

B. How these results were reached

We set out to compare two brokers on a live sports feed. The audit of Section III-D rejected our own first answer, a complete and statistically significant result set; a later audit of the benchmark refuted a claim we had already made about it. What follows reports what survived. The order of discovery, the withdrawn claims and the failed pre-registrations are recorded in the supplementary material, because the credibility of an audit depends on its own failures being visible; they are not the argument and no longer organise the paper.

II. Background and Related Work

The claim that measurement can dominate a benchmark is not new; what is missing is the practice of checking for it run by run.

A. Streaming benchmarks

Broker comparisons are published continuously, in peer-reviewed venues [2] and in grey literature that practitioners actually read. The two systems compared here, Apache Kafka and Redis Streams, are the pair those comparisons most often name, and the OpenMessaging Benchmark [3] is the community's shared instrument. That experimental setup can dominate a result is established [4], [5], and how to report one without misleading is a settled literature we follow rather than restate [6], [7], [8]; Schroeder et al. [9] give the precedent for our result's shape: a choice the experimenter regards as incidental determines the answer.

The stream-processing benchmark literature has already thought carefully about where to stand a clock. Karimov et al. [10] separate event-time from processing-time latency and keep the driver off the machines under test; Van Dongen and Van den Poel [11] use a single broker and its append timestamps "to ensure correct latency measurements", which is our one-clock rule arrived at independently; Fruth et al. [12] show a harness recording latencies imposed by its own execution environment. What that literature does not do is ask what a millisecond stamp does to a sub-millisecond path, or check the sample the instrument keeps. Those two questions are this paper.

B. Measuring time across processes

Lamport [13] established that a distributed system has no single physical time, only a causal partial order, and that order matters here in a way this work gets right only in the present version: an acknowledgement received by the producer and a record received by the consumer are two branches descending from the broker's append, so neither precedes the other. The engineering problem of agreeing clocks is well known [14], [15], and instruments built to beat it exist. Cloudprofiler [16] calibrates cross-node counters to $\sim 92\,\mu$s precisely because millisecond synchronisation is too coarse for streaming engines; Lancet [17] uses NIC timestamps, verifies statistically that the load it applied is the load it intended, and refuses to report a tail it cannot stand behind; P4STA [18] moves stamping into the switch dataplane. The accuracy of the timer itself is a measured quantity [19], [20].

We therefore claim less than "nobody has considered this", and one prior result bounds the claim closely enough to state outright. Villain et al. [21], profiling FreeBSD against a DAG hardware reference, found that the outgoing socket timestamp "does not respect causality" — packets leave before the stamp meant to mark their departure is written — and that its error cannot be bounded; they observe it under every stress pattern including none, and attribute host latency generally to interrupt processing, scheduling and context switching. That is Mode A's physical primitive, published in 2012 and measured more precisely than we measure it. What we add is not the primitive but its consequence for a two-process measurement: a span whose endpoints are written by two different threads, a probability law relating the inversion rate to the interval being measured (Equation 3), and manipulations that separate scheduling from every rival explanation. The audit half has its own ancestor: Paxson's calibration study [1] treated physically impossible transit times as evidence that an instrument had failed and reported the proportion of traces affected. What is missing in both cases is that practice applied uniformly, run by run, to a modern streaming benchmark, with the rejection rate published the way a survey reports its response rate.

Concurrent work reaches the same observable by a different route. In a 2026 preprint, Sharma et al. [22] count negative timing spans in distributed inference systems and propose a `causality_health` check; injecting skew, they observe no violations up to 3 ms and clear violations by 5 ms, and read that as a threshold in skew. We contradict a premise of that reading rather than their measurements. They hold that "queueing alone cannot produce negative timing spans or cause timestamps to imply reversed causal orderings"; we observe inversion rates near 23% at 88% utilisation with no skew available to blame, on one host and with an inter-host offset of 0.067 ms where two are used, and Section V moves that rate fiftyfold without touching a clock. Scheduling is a third channel, so a threshold expressed in milliseconds of skew does not transfer to a loaded machine. A companion 2026 preprint [23] pre-registers its measurement protocol, as we do here.

C. Resolution and discarded samples

That a timer coarser than the interval invalidates the interval is textbook [19], [24], and metrology has carried both of our failure modes in a single uncertainty budget for as long as it has calibrated stopwatches: the NIST guide lists the operator's reaction time — a stall between the event and the stamp — beside the display resolution, and notes that at short intervals the resolution becomes significant against the instrument's rated accuracy [25]. Our contribution to that pairing is not the pairing. That periodic sampling commensurate with a periodic phenomenon observes only part of that phenomenon is stated in the IPPM framework [26], which recommends randomised probe timing for exactly that reason, and RFC 3432 makes a randomised start mandatory for periodic streams [27]. Coordinated omission [28] is the mirror image of what we report: there the instrument fails to record samples that would have been slow, here it deletes samples that were fast, and Tene's general point, that omission correlated with the value being measured skews the result, covers both directions. The precondition has been seen before in another benchmark: YCSB reported sub-millisecond operations as zero milliseconds until it adopted microsecond histograms [29], and it kept those samples rather than deleting them. Dither is signal processing's standard cure for a periodic sampling artefact [30], and the underlying equidistribution mathematics is a century old. What we claim as new is not quantisation but its *interaction* with a few-phase producer and a positivity guard, and the law that interaction obeys.

D. Where scheduling delay comes from

A thread that is runnable does not always run. Work-conservation violations are documented for CFS-era schedulers [31]; ours is an EEVDF-era kernel. Queueing gives the leading-order account of waiting, and predicts correctly that utilisation alone does not fix a waiting time — pooling and geometry matter at fixed ρ — so the geometry contrast of Table IV refutes a single-parameter law rather than queueing itself. A 2026 preprint by Chandrasekar and Kramberger [32] applies exactly such a single-parameter M/G/1 framing to benchmark bias, which we adopt as a leading-order account and, on the load axis, ultimately refute (Section V). Tail behaviour under load [33] is the phenomenon our stall distribution samples, and that work also shows real-time priority collapsing a tail that renicing leaves alone — on the system under test, where we apply the same lever to the instrument.

III. Method

Everything below is measured on one clock unless the text says otherwise, and the spans we audit are named explicitly, because which two stamps are subtracted is the whole question.

A. Testbeds

Every finding comes from four Oracle Cloud `VM.Standard.E5.Flex` instances (three brokers, one driver), Ubuntu 22.04 on kernel `6.8.0-1057-oracle` (EEVDF scheduler), CPython 3.10, a private network, client libraries pinned identically, and `chrony`-disciplined clocks logged every minute. Measurement perturbs, and one early phase perturbed enough to be unusable: it pinned the load generator to a subset of cores while utilisation was measured across all of them, which dilutes ρ and makes that phase incomparable with the rest. It is excluded from every result reported here, and the campaigns that remain measure utilisation with no core pinning. A second testbed, a single workstation running the brokers under containers, produced our withdrawn first result and is reported only as an audit outcome.

B. The measurement at a glance

The harness runs producer and consumer as separate *processes on one host*, both stamping the wall clock (`time.time_ns()`, CLOCK_REALTIME), not a monotonic counter or the TSC, since a cross-process span needs a shared epoch that no per-core cycle counter provides. The workload is a public football event feed (StatsBomb open data, CC BY-NC-SA 4.0) replayed against Kafka and Redis Streams. The send schedule is measured rather than assumed: pacer jitter is 67–69 μs at p90, and the achieved rate is recorded per run against elapsed wall time. The measured inter-host offset, where two hosts are used, is 0.067 ms. The primary measure is **Time-to-Insight** (TTI), from scheduled emission to availability at the consumer:

$$\mathrm{TTI} = \underbrace{(t_{\mathrm{send}} - t_{\mathrm{sched}})}_{\text{scheduling lag}} + \underbrace{(t_{\mathrm{recv}} - t_{\mathrm{ack}})}_{\text{transport proxy}} + \text{residual}. \quad (1)$$

C. One component is a proxy, not a chain

This distinction is load-bearing, and earlier versions of this work did not draw it. The transport term of Equation 1 subtracts the producer's receipt of the broker's acknowledgement from the consumer's receipt of the record. The broker's append precedes both of those events, but neither of them precedes the other: they are two branches of one cause. The acknowledgement stamp is therefore a *late, producer-side observation* of a broker-side event, and when the callback branch runs slower than the delivery branch the difference is negative without anything physically impossible having occurred. By contrast $t_{\mathrm{recv}} - t_{\mathrm{send}}$ and TTI itself are genuine chains: the producer sends, the broker appends, the consumer receives. Section V-A reports all four spans, and they behave completely differently.

D. The consistency check

We fix the rule before the final campaign and apply it to every run, including those whose results looked reasonable. **A run is rejected if more than 1% of its events carry a negative value in any latency component, or if the median of any component is negative.** A condition is usable only if all of its runs survive; where partly rejected we report the retention rate. The one-per-cent figure is a threshold, not a discovery: the supplementary material sweeps it from 0 to 20%, and no condition behind our first result is fully usable at any value in that range, so the withdrawal does not turn on where the line was drawn.

The justification is not that a negative is impossible, because for the transport proxy it is not. It is that a negative proves the reference stamp cannot serve as the origin of that event's interval. A run whose reference is unusable on more than one event in a hundred cannot report a latency, whatever the cause. The check is a *necessary* condition, not a validation procedure: a corpus that fails is unusable, and one that passes has cleared the cheapest available bar and nothing more. We use the sign as a gate rather than as an estimator, although the same constraint supports estimation, since offset and skew can be recovered from a delay envelope [1]. That was deliberate:

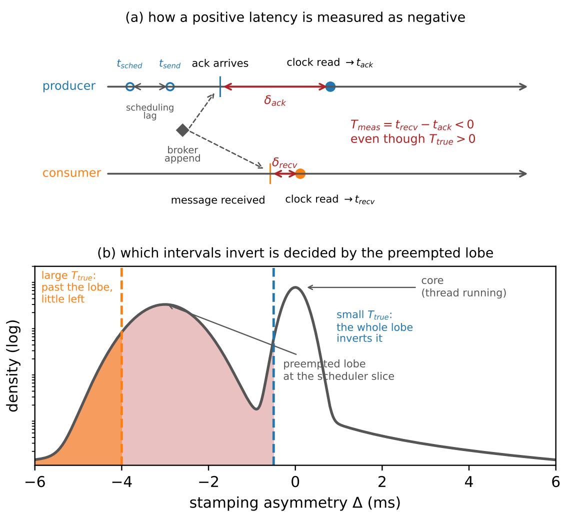

Fig. 1. (a) How a positive latency is measured as negative, on one clock: each side stamps some interval after its event, and if the producer's callback delay exceeds the true delivery plus the consumer's delay, the difference inverts. The record is not early; the acknowledgement stamp is late. The broker's append is the common cause of both arrivals — they are two branches, and neither precedes the other, which is why their difference may be negative with nothing physically impossible having happened. The producer's own stamps, t_{sched} and t_{send}, do precede the acknowledgement, and the span referenced to them never inverts (Table III). (b) Why the check bites hardest on small effects, drawn schematically with the measured mode's position: the stamping asymmetry Δ has a narrow core when the callback thread is running and a lobe near -3 ms when it is not, so a small T_{true} is inverted by the whole lobe and a large one only by what lies beyond it.

an estimator corrects a record, a gate declines to publish from a run whose instrument demonstrably failed, and after this check destroyed our own first result we preferred declining.

E. A model of the failure

Every timestamp is taken some interval after the physical event it claims to mark, because the thread that reads the clock must first be scheduled. This is not the cost of the clock call, which is small and separately measurable [34]; it is the delay before the call runs at all. Writing δ for those stamping delays,

$$T_{\text{measured}} = T_{\text{true}} + \Delta, \qquad \Delta \equiv \delta_{\text{recv}} - \delta_{\text{ack}}, \tag{2}$$

so Equation 2's difference inverts exactly when $\Delta < -T_{\text{true}}$, giving

$$\Pr[\text{inversion}] = F_{\Delta}(-T_{\text{true}}). \tag{3}$$

Equation 3 says the most useful thing in this paper (Fig. 1b). **Inversion probability is a property of the quantity being measured, not only of the machine**: a fixed distribution of Δ overlaps a small T_{true} almost entirely and a large one hardly at all, which is why our network-delay arm, transport in tens of seconds, passes every run while same-hardware arms measuring one millisecond fail wholesale.

F. Statistics

Every proportion below is a count over a stated denominator and carries a Wilson score 95% interval computed from that denominator, Wilson rather than Wald because at the real-time arms' rates the normal approximation places the lower bound below zero. Every ratio between two proportions is accompanied by a two-proportion z with pooled variance, and we state whether the two intervals are disjoint. Location differences use Hodges–Lehmann [35] shifts with distribution-free intervals and rank-sum tests; equivalence claims use TOST [36] with the margin stated and the interval printed; Holm [37] corrects within a named family, and the corrected value is the one the tables display; percentile bootstrap intervals cover statistics with no closed form. The grid-membership inference uses a permutation null, described where it is used. Regression intervals are Student-t at $n-2$ degrees of freedom. Tail indices are estimated on the traced histogram by grouped maximum likelihood with a profile-likelihood interval and by an exceedance ratio with a Wilson interval, because the buckets are interval-censored and a slope through survival points is not an estimator; the grouped estimator on log-spaced bins is Virkar and Clauset's, which is the Hill estimator for binned data, and we use their bootstrap for goodness of fit [38]. All interval arithmetic is recomputed from the committed artefacts at build time by `scripts/stat_intervals.py` and `scripts/tail_index_traced.py`, and the tables reporting it are generated rather than transcribed.

IV. Mode B: A Benchmark That Deletes Its Own Samples

The most widely used cross-broker benchmark computes its reported latency distribution from a subset of the samples it collected, does not record how large that subset was, and the size of the subset is determined by the send rate. Everything measured below is the OpenMessaging Benchmark in embedded mode, where driver and workers run in one JVM; its distributed mode is discussed in Section VII-D.

A. What the benchmark does, and why

In `LocalWorker.java`:294 the end-to-end latency is `now - publishTimestamp`, where `now` is read in the consumer and `publishTimestamp` is Kafka's producer-set `CreateTime` [39]. In `WorkerStats.java`:95 the sample enters the histogram only `if (endToEndLatencyMicros > 0)`. The message is counted as received one line earlier, but a non-positive latency is dropped and no counter records how many.

The guard's origin is documented and its reasoning is sound. It was added in 2018 to stop negative values reaching the histogram, on the ground that end-to-end latency "is calculated based on time difference of `System.currentTimeMillis()` on two different machines" and "can be negative" [40], which is the line quoted above; the recorder is an `HdrHistogram Recorder`, which rejects negative values outright. Filtering for positives is the reasonable local fix, and defensive rather than evasive: the author guarded the recorder against an input it rejects. Its non-local consequences are two, neither considered at the time. First, `> 0` also deletes every sample that computes to exactly zero, and because `System.currentTimeMillis()` has millisecond resolution, any delivery faster than one millisecond does compute to zero; on co-located paths, where our own transport measures 0.1–0.5 ms, that is most of them. Second, the one observation which would prove the instrument had failed is the exact observation the guard removes. The convention is institutional rather than incidental: KIP-489 codifies reporting a negatively computed latency as `NaN` [41], also uncounted.

The guard is not the only silent loss this harness has carried. A serialisation defect found in 2023 truncated the histogram on long runs and discarded most of the latency data, again with nothing in the output to show it [42]. That one was a bug and was fixed; the guard is a design choice, which is why it is still there.

B. The consequence, measured

Of the 75 instrumented cells whose own summary we captured, 71 report a median of 1.0 or 2.0 ms, and across those the benchmark computes its reported distribution from between $\mathbf{0.36}\%$ and $\mathbf{100}\%$ of the samples it takes. Two replicates of one cell — 200 B payload at 500 msg/s, same host — kept 431 and 120,434 samples, a 279-fold difference, and both reported a median of 1.0 ms with nothing in the output to distinguish them. Three uninstrumented runs print p50, p95, p99 and max all equal to 1.0 to three decimal places. The full ledger holds 223 instrumented runs, 11,532,492 discarded samples and 41,403 negatives, and over it the retained fraction falls as low as 0.0044% — four samples kept of ninety thousand.

C. The mechanism is phase, not speed

Publish latency across these cells takes only two values, 0.3 and 0.4 ms, while retention varies 279-fold. A two-valued predictor cannot account for a range like that whatever its correlation, and the correlation it does have (+0.31 over 71 cells) carries the sign Equation 4 predicts. What moves retention is where the producer's send instants fall against the millisecond grid. A sample survives when its delivery crosses a tick boundary, which for a delivery shorter than one tick happens with probability T_{true}/τ:

$$\text{retention} \;=\; \min\!\left(1,\; \frac{T_{\mathrm{true}}}{\tau}\right) \qquad \text{(uniform phases).} \tag{4}$$

Both incommensurate arms sit near 50%, implying $T_{\mathrm{true}} \approx 0.5$ ms at $\tau = 1$ ms, which agrees with our own unquantised transport measurement.

D. Commensurability is not binary

Phases are not uniform when the send interval is commensurate with the tick. Write the send interval over the tick in lowest terms as $\Delta/\tau = p/q$ with $\gcd(p, q) = 1$. After q sends the phase returns to its start, so a producer visits exactly q distinct phases, and a single run lands on one of the two grid points bracketing T_{true}/τ: $q = 1$ is the all-or-nothing corner, an incommensurate rate is effectively continuous, and everything between is a grid, coarser meaning less reproducible. The replicate spread follows:

$$\text{spread} \;=\; \begin{cases} 100/q & \text{when } T_{\mathrm{true}}/\tau \text{ sits mid-cell,} \\ 0 & \text{when } T_{\mathrm{true}}/\tau \text{ sits on a grid point.} \end{cases} \tag{5}$$

TABLE I

Replicate spread against the phase denominator q, at fixed payload, load, duration and host. *Position* is the distance from the measured continuous value to the nearest grid point, against the cell half-width: 0 sits on a grid point, 1 midway between two. Position is scale-free, so every call is unchanged whether the continuous value is estimated pooled or per rate. †Already in lowest terms.

rate	Δ/τ	q	n	width	pos.	predicted	spread
1000/s	1/1	1	3	100.0	0.99	full	99.3
500/s	2/1	1	4	100.0	0.99	full	99.5
250/s	4/1	1	3	100.0	0.99	full	58.6
400/s	5/2	2	5	50.0	0.02	flat	17.6
300/s	10/3	3	5	33.3	0.97	full	30.5
600/s	5/3	3	5	33.3	0.97	full	32.7
800/s	5/4	4	5	25.0	0.04	flat	7.2
625/s	8/5	5	5	20.0	0.95	full	17.9
875/s	8/7	7	5	14.3	0.93	full	10.7
incommensurate — phase set effectively continuous							
889/s	1000/889†	>64	5	—	—	≈ 0	1.7
457/s	—	>64	4	—	—	≈ 0	2.1
383/s	—	>64	4	—	—	≈ 0	2.8
333/s	—	>64	3	—	—	≈ 0	3.1
611/s	—	>64	3	—	—	≈ 0	3.1

Table I reports every arm. A round number is the worst possible choice: 500 msg/s is exactly 2.000 ms, $q = 1$, and its four replicates retain 0.47, 1.51, 18.02 and 99.99 per cent. At 457 and 383 msg/s, incommensurate, replicates hold to 2.1 and 2.8 points despite intervals differing by 19%. The exactness matters: 889 msg/s sits 0.00014 from 9/8 and behaves as fully continuous, while 300 msg/s spreads 30.5.

E. The grid as inference, not description

A table of spreads is not a test, so we state one. For each arm let D be the mean distance from its replicates to the nearest vertex of the q-grid, normalised by the cell half-width. Under the continuum null, retention is a continuous function of rate and replicate positions are uniform on the cell; the null distribution is obtained by permuting replicate positions within the arm (10^4 permutations, so the smallest attainable p is 2.5×10^{-4}). Table II reports every arm with $n \geq 5$ and prints the corrected value, not the raw one. 9 of the 10 powered arms reject the continuum null at $\alpha = 0.05$ after Holm correction within the family of 12. The tenth, 900 msg/s, rejects before correction ($p = 0.044$) and not after ($p = 0.131$); we report it as unresolved rather than as support, which an earlier version of this table did not. The remaining 2 arms, 400 and 800 msg/s, have no power by construction: when T_{true}/τ sits on a vertex the grid prediction and the continuum prediction coincide, so no test can separate them. A model that claimed separation everywhere would be the more suspicious one.

TABLE II

Grid membership against a continuum null. D is the mean normalised distance to the nearest grid vertex; under the null its expectation is the value in the *null* column. p_{Holm} is a permutation p-value (10^4 draws) after Holm correction within the family of all 12 arms; the corrected value is what decides the verdict. Arms marked *flat (coincident)* are those the model predicts flat, where the two hypotheses make the same prediction and the test has no power. This table is generated from `GRID_MEMBERSHIP.CSV` at build time.

rate	p/q	n	D	null	p_{Holm}	verdict
1250/s	4/5	5	0.131	0.351	0.003	grid
1000/s	1/1	5	0.003	0.479	0.003	grid
900/s	10/9	5	0.265	0.347	0.131	not resolved
875/s	8/7	5	0.209	0.388	0.003	grid
800/s	5/4	5	0.104	0.052	1.000	flat (coincident)
700/s	10/7	5	0.329	0.438	0.009	grid
625/s	8/5	10	0.112	0.470	0.003	grid
600/s	5/3	6	0.034	0.485	0.003	grid
500/s	2/1	5	0.043	0.499	0.003	grid
400/s	5/2	5	0.093	0.006	1.000	flat (coincident)
300/s	10/3	10	0.221	0.482	0.003	grid
250/s	4/1	5	0.181	0.494	0.003	grid

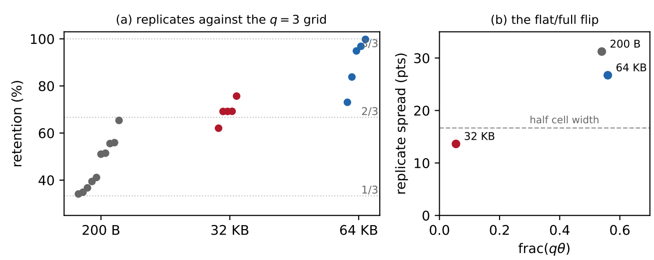

Fig. 2. The campaign's confirmed prediction. (a) Retention replicates at 300 msg/s ($q = 3$) by payload, against the thirds grid: 200 B straddles both branches, 32 KB pins near the $2/3$ vertex, 64 KB re-opens to the upper cell. (b) Replicate spread against $\mathrm{frac}(q\theta)$: the flat–full boundary at half the cell width is crossed in both directions by the one manipulated variable.

F. The confirmation manipulates the physics

Payload moves T_{true}, hence the grid position, and Equation 5 dictates the arm's class with nothing else changed. The prediction was registered before the runs, and it came true (Fig. 2): spread collapses to 13.6 at 32 KB, three replicates within 0.06 of each other, pinned near the $2/3$ vertex, and re-opens to 26.7 at 64 KB, the flat–full boundary crossed in both directions by one manipulated variable. One registered detail failed: the 32 KB pin sits at 69.2, not the predicted 66.67.

A comprehensive overnight campaign ran 63 cells at $n \geq 5$, with six predictions and their falsifiers registered in advance; every cell completed validly. One prediction was confirmed, three falsifiers fired, one was not evaluable and one split. A cumulative drift account is falsified by duration invariance: intermediate counts of 1, 1 and 0 over one, three and ten minutes, with vertices pinned throughout, so what stands is per-send jitter rather than accumulation. Incommensurate arms at 1053 and 1219 msg/s measure 47.2 and 48.2%, refuting the linear extrapolation one registered prediction assumed.

G. Generality, and the sign channel

To show the law is not an artefact of one JVM, we reproduced the guard in an independent Python harness sharing no code with the benchmark. On one clock, the harness recorded **zero** negative differences across 905,040 samples, as a causal chain requires, while the grid behaviour reappeared intact. Across two `chrony`-disciplined hosts, retention wandered monotonically from 13.4 to 27.0% over four replicates where the same arm on one clock held to 0.8 points, offset drift entering T_{true}; so cross-host the deleted fraction couples to the run-time synchronisation state, and nothing in the output records either.

Separating the discards by sign is what makes the guard's cost legible. Across all 223 runs the Kafka-driver corpus's 10,913,263 discarded samples contain **not one negative**: the most negative value at any load, size or rate is $0\,\mu$s. Those discards are resolution failures, not clock failures, a distinction an unsigned counter cannot make and one an earlier version of this work got wrong. The same channel then caught the real thing in the Redis-driver replication: 41,403 negatives, every one exactly $-1000\,\mu$s, the minimum the quantum can express, with 41,397 of them in a single run where they were *half of all samples taken*, beside six kept.

That value is not a coincidence, and the driver explains it. The Redis driver takes its publish timestamp from `entry.getID().getTime()` in `RedisBenchmarkConsumer.java`:72 — the stream-entry identifier the *Redis server* assigns on `XADD` [3] — whereas the Kafka driver uses a producer-set stamp. The Redis end-to-end span is therefore a difference between two millisecond-floored clocks on two hosts, and a sub-tick offset between two floored clocks can produce exactly one negative value, $-1000\,\mu$s, and no other. That is precisely what the corpus contains. It also explains the contrast that makes the sign channel legible: the Kafka-driver span floors two stamps whose difference is taken inside one JVM, and it never goes negative at all. Clock discipline was clean at every logged minute, which is the point — this failure needs no misbehaving clock, only two of them and a floor. The same guard that deletes your fastest measurements will also delete the evidence that your instrument spans two clocks, and the report looks identical either way.

H. Would finer timestamps fix it?

Partly, and the part they miss is the more dangerous one. A microsecond-grained stamp makes $T_{\mathrm{true}}/\tau \gg 1$ on these paths, so by Equation 4 the deletions that come from differences collapsing to zero disappear, and the grid arithmetic recedes to paths faster than the new quantum. The positivity guard survives untouched. Genuine negatives still occur at microsecond resolution, from stamping stalls (Section V), from the $\approx$70 μs of inter-host offset, and from virtualised-clock artefacts, and the guard would go on deleting them silently, converting an uncounted *resolution* failure into an uncounted *instrument* failure.

TABLE III
NEGATIVES BY SPAN, OVER EVERY ARCHIVED RUN THAT JOINS PRODUCER AND CONSUMER EVENTS (5,913 RUNS, 738,730 EVENTS). ONLY THE SPAN WHOSE ORIGIN IS AN ACKNOWLEDGEMENT INVERTS. THE OTHERS ARE CAUSAL CHAINS AND NEVER DO.

Span	Origin stamp	Negatives	Share
$t_{\rm recv} - t_{\rm ack}$	acknowledgement (proxy)	62,264	8.43%
$t_{\rm recv} - t_{\rm send}$	send (causal chain)	0	0%
$t_{\rm out} - t_{\rm send}$	send (causal chain)	0	0%
TTI	scheduled emission	0	0%

V. MODE A: WHY A LATENCY COMES OUT NEGATIVE

The negatives in our own corpus are produced by scheduling, and we establish this by moving scheduling while holding everything else fixed.

A. What the audit rejected

Applying the rule of Section III-D to all 2,266 runs rejects 1,321: 862 of 1,382 on the workstation testbed (62.4%) and 459 of 884 on the cloud testbed (51.9%). Every run behind our first result is among them. At saturation Kafka's median transport proxy reads -6.4 ms at one hundred connections.

Table III separates the components over the archived corpus. The acknowledgement-referenced proxy is negative on 62,264 of 738,730 events (8.43%; the Wilson interval is narrower than 0.1 points at this denominator, and we omit it hereafter), and 3,976 runs exceed the one-per-cent threshold on it. The send-referenced span and TTI, which are causal chains, are negative on **no event at all**. Nothing in this corpus arrived before it was sent; the reference stamp was late. The rejections are therefore evidence of an unusable reference rather than of impossible physics, which is precisely what the gate is for.

B. Why it looked correct

The rejected result was monotone in concurrency, significant after correction ($p = 9.0 \times 10^{-11}$), and had a tidy architectural story. It passed every conventional check we knew: replicate counts, effect sizes, corrected significance, sanity plots. The acknowledgement timestamp is taken by the producer's callback thread; accelerated replay saturates the CPU, the thread is descheduled, and by the time it reads the clock the consumer has already recorded receipt. That is a timestamping race under load, which is why medians stay positive at moderate load and invert wholesale only at saturation.

C. What the check cannot catch

A great deal. Load-induced inflation that stays positive is physically possible and needs a better instrument to detect. Our own second withdrawal is the example: a twentyfold end-to-end gap that was a per-run start-up cost read as a per-event constant, entirely causally consistent and entirely wrong. Causal consistency is necessary, not sufficient.

TABLE IV
THE MECHANISM, BY MANIPULATION (2,985 MATCHED EVENTS PER CELL; WILSON 95% INTERVALS IN BRACKETS). TOP: STAMPING THREADS MOVED TO `SCHED_FIFO` WITH BACKGROUND LOAD UNCHANGED; ACHIEVED UTILISATION MATCHES TO 0.001 BETWEEN ARMS, SO OCCUPANCY ALONE MOVED. BOTTOM: IDENTICAL UTILISATION REACHED BY TWO CORE GEOMETRIES ($\rho = 0.7531$ IN ALL FOUR k=6 ARMS); A FUNCTION OF ρ RETURNS ONE VALUE FOR ONE INPUT.

Manipulation	Arm	Rate	95% CI	Factor (z)
Priority, 75%	ordinary	0.1320	0.1203–0.1446	
	real-time	0.0034	0.0018–0.0062	39× (19.8)
Priority, 88%	ordinary	0.3049	0.2886–0.3216	
	real-time	0.0057	0.0036–0.0091	54× (31.9)
Geometry, original	concentrated	0.0824	0.0731–0.0928	
	spread	0.1709	0.1578–0.1848	2.07× (10.3)
Geometry, repl.	concentrated	0.0600	0.0520–0.0691	
	spread	0.1229	0.1117–0.1352	2.05× (8.4)

D. A rare state, not a wide distribution

The stamping thread is either running, in which case the stamp is late by a narrow jitter of width σ_c, or preempted and off-CPU. Writing $p(\rho)$ for the probability of the second state and $S(\cdot)$ for the residual's survival function,

$$\Pr[\text{inversion} \mid T_{\rm true}] \;=\; p(\rho)\; S(T_{\rm true}). \tag{6}$$

Load changes how *often* the thread that must record the time is not running, not how wide the ordinary jitter is. Going from idle to the knee multiplies the fitted core width σ_c by 5 and the inversion rate — the mass of Δ beyond $-T_{\rm true}$ — by 61. The rate has a ceiling below one: fully saturated it reaches 0.37, so even then most events are stamped either unpreempted or by a thread whose stall is shorter than the interval being measured.

E. The experiments that settle it

Real-time priority on the stamping threads moves occupancy while utilisation stays fixed (Table IV, top). The rate falls by factors of 39 and 54; the Wilson intervals of the two arms are disjoint at both load levels ($z = 19.8$ and $z = 31.9$), and both residual rates land on the floor the model names in advance, $C_0 \approx 0.004$. *This refutes every account in which the inversion rate is a function of utilisation*, including the queueing form we ourselves adopted: ρ did not change and the rate changed fiftyfold. Across eight matched pairs spanning 60 to 95% load the factor ranges 7–80×.

Three further manipulations agree. At $k = 6$ both load geometries sit at $\rho = 0.7531$, identical to four decimals, and the inversion rates differ by 2.07× ($z = 10.3$), reproduced at 2.05× by a full replication. Multi-server queueing expects geometry to matter at fixed ρ, so this will not surprise a queueing-literate reader; what it rules out is the single-parameter form we had adopted and pre-registered, in which ρ alone fixes the rate. Payload padding lengthened transport 76.9× while the inversion rate *fell* 4.1×, monotonically, with utilisation held to a 0.012 spread: the direction Equation 3 predicts and the opposite of a stress account. Finally, measured with `bpftrace` on the `sched_wakeup` and `sched_switch` tracepoints [43], filtered to the harness processes and sharing no instrument, estimator or data with the application-level rate, the probability that a run-queue stall outlasts the interval agrees with the observed inversion rate across three arms at two load levels, ratios 0.78, 1.06 and 1.32 with no consistent sign: an application-level failure rate predicted to within a third, unfitted. A probe on every context switch is not free, and we measured what it costs by running the same cell untraced: the inversion rate is 0.272 untraced against 0.231 traced ($z = 3.6$). The comparison above is against the traced arm's own rate, so the agreement is between two instruments observing one machine; the untraced rate is the one to quote for the machine itself.

F. Both load-axis predictions failed, including ours

The knee sweep placed conditions where the candidate load laws diverge. The M/G/1 form fitted *worse than the mean* ($R^2 = -0.05$ against a fitted exponential's 0.93), and our own registered bracket missed a $1.44\times$ growth by predicting 2.45–$3.07\times$. Both were parameterised in ρ, and ρ is not the variable. The Pollaczek–Khinchine form remains motivation rather than fitted result.

G. The stall distribution has a mode at the scheduler's slice

It is not a heavy tail. It is three modes, and the one that matters sits exactly where the kernel's own constant puts it.

Over a $77\times$ payload span the inversion rate falls monotonically with T_{true}, and log–log least squares over the four payload levels returns an effective exponent of 0.339 (0.234–0.443, Student-t at two degrees of freedom). That fit is in the supplementary material, because four points do not earn an equation here. An earlier version claimed the kernel trace confirmed it independently. It does not, and we withdraw the claim; estimating the traced index properly is what refuted it, and what the trace says instead is more useful.

Over the 551,956 traced wakeups the grouped maximum-likelihood index on 0.25–2 ms is 1.19 (1.18–1.20) while the exceedance index on 0.5–2 ms is 0.21 (0.20–0.21). Two estimators of one quantity cannot differ sixfold unless the model is wrong, and a bootstrap goodness-of-fit says it is: $p < 0.0004$ over 2,500 replicates drawn from the fitted power law itself. The reason is visible without any fitting. The counts do not decrease monotonically, as a power law must: they have 3 local maxima, and the last is a *mode* at 2–4 ms holding 10.5% of all wakeups, $4.5\times$ its lower neighbour. Above it the survival is well behaved and light — grouped maximum likelihood beyond 4 ms gives 2.04 (2.00–2.07), a finite variance.

That mode is the preempted state of Equation 6, observed directly and at the scale the scheduler sets. The constants are *derived*, not measured: they were not captured during the campaign, so we take them from the published kernel package and from the campaign's own data rather than from a machine only we can log into. The online CPU count follows from the geometry conditions themselves, since each loads k cores and records the achieved utilisation, and k/ρ recovers 8 six times over with a spread of 0.05. The kernel's rule then gives a base slice of $4 \times 0.75\,\mathrm{ms} = 3$ ms, and the build sets `CONFIG_HZ`=1000, a 1 ms tick. With run-to-parity a woken thread that loses the wakeup-preemption check waits for the incumbent to exhaust that slice, and expiry is noticed at the tick. A thread descheduled at the wrong instant therefore reads its clock about a slice late, which on a path whose true transport is 0.1–0.5 ms is six to thirty times the interval being measured. No scheduler tunable is written anywhere in the archived campaign, which is why the derived value is the one that applied; that is strong evidence rather than proof, and we report it as such. The practical consequence is unchanged and now has a cause: a mean over this distribution is dominated by a mode the operator never sees, so a mean-based counter is a poor instrument for this failure and comparing mean against median cannot flag it.

H. Scope, and the alternatives eliminated

The mechanism requires a stamping path that can be preempted. Designs that pin a thread to a dedicated core and busy-poll, or that stamp in hardware, remove the state Equation 6 depends on, and we make no claim about them. Within our own setting the alternatives are each eliminated: clock skew (one clock by construction on the rejecting arms; zero negatives in 905,040 deliberate two-clock samples); utilisation (identical ρ, rates twofold apart); generic timer noise (`SCHED_FIFO` at fixed utilisation collapses the rate 39–54×, and noise does not respect scheduling class); stress (transport 77× longer *lowers* the rate); and direct observation (a `sched_switch` histogram predicts the rate to within a third, unfitted).

VI. What the Brokers Actually Do

Once the failed runs are removed, the broker comparison that motivated the work is small, and one configuration choice is far larger than it.

On gated artefacts the two brokers sit within a millisecond of each other: the Hodges–Lehmann shift is 0.408 ms (90% interval 0.389–0.419), a TOST against a 1 ms margin rejects both one-sided nulls at $p < 0.001$, and the shift stays between 0.408–0.417 ms across the 3 concurrency levels tested, so there is no degradation with concurrency over that range. This comparison is on the transport proxy, which Section III-C warns is not a causal chain; it is serviceable here because these runs cleared the gate, and TTI, which is a chain, is equivalent at the same concurrency levels against a wider margin (supplementary material). It is not a purchasing argument, and it does not survive the failure modes above with enough precision to become one.

One condition reverses the ordering, and it is not the broker. Injecting one-way delay identically at both brokers reverses them completely: Kafka tracks the injected delay almost additively, while Redis collapses, its median transport reaching 4.6 s at 5 ms of injected delay, 31.3 s at 20 ms and 102.5 s at 50 ms over five feeds per arm, roughly 900 to 2,050 times the delay applied. The cause is the consumption pattern rather than the broker: a per-message acknowledgement turns each event into a sequential round trip, so ordinary network delay becomes seconds of staleness, while a prefetching consumer serves from buffer with no per-record round trip to expose. The account was given a falsifier in advance — if amortising the acknowledgements does not help, it is wrong — and batching them in groups of two hundred cuts the median from 4,138 ms to 103 ms, a factor of 40, replicated four times.

Each broker therefore has one client-side setting worth one to two orders of magnitude, and both are invisible on a co-located testbed, because the round trip they multiply is too short there to matter. A benchmark run on loopback cannot see either. That is the finding worth carrying: not that settings matter, which vendors document, but that the standard evaluation environment structurally conceals the ones that matter most.

VII. Discussion

A. For benchmark authors

Report a physical-consistency audit. Any cross-process timestamp difference admits a sign check that costs nothing; it rejected 58% of our own runs, every one having passed every conventional check we knew. Better instruments exist [16], [17], [18]; whatever the instrument, check its output against causality and publish the rejection rate, as was done for network clocks nearly thirty years ago [1].

Name the span you are subtracting. A difference between two stamps is a latency only if the first event causally precedes the second. An acknowledgement received by the producer does not precede a record received by the consumer; both descend from the broker's append. Our own text made this mistake for two versions, and the corpus caught it only when we counted the send-referenced span separately (Table III).

Count your events before you quote a percentile. Our second withdrawal came from reporting a median over seven events per run as a property of the system; it survived a hundred runs because a deterministic per-run artefact reproduces beautifully.

Make the timestamping path unpreemptable, and say that you did. The one mitigation our measurements support directly: real-time priority on the stamping threads cut the inversion rate 7 to 80$\times$ at unchanged background load. Busy-polling on a dedicated core should buy the same effect by the same means, at the price of a burned core. Two caveats: the floor is not zero, so the check stays; and a real-time or spinning stamping thread changes the system it measures, so where that is unacceptable, report the retention rate instead.

Dither the send instant, and publish the retention rate. Randomising probe timing to avoid synchronising with a periodic phenomenon is long-standing advice: profilers randomised their sampling clock for it thirty years ago, the IPPM framework recommends it [26] and RFC 3432 requires it of periodic streams [27]; Lancet [17] already verifies statistically that the load applied is the load intended. What we add is the reason it matters at this scale: the phenomenon being synchronised with is the timestamp quantum itself. Publish beside the latency, because neither is recoverable afterwards, the retention rate — a summary from 0.36% of samples is typographically identical to one from all — the timestamp resolution, and the synchronisation state. Active network measurement has done exactly this since 2006: OWAMP attaches a synchronisation flag and an error estimate to every timestamp it reports, and requires that a packet stamped in the future be recorded unaltered rather than dropped [44]. We ask for no more than that, applied where brokers and language runtimes report latency, and where nobody does it. Count discards separated by sign: negative means the reference stamp failed, computes-to-zero means the resolution failed, and a single unsigned total cannot distinguish them [45]. We drew a wrong conclusion from one that could not.

Record the achieved rate, not the flag meant to produce it. Verify it per run against elapsed wall time. The send schedule is an experimental variable in Mode B, not a nuisance parameter, and we found runs where the configured and achieved rates disagreed.

B. A better clock does not reach the failure

Hardware-timestamped PTP [14] would take two LAN hosts from our $\approx 70\,\mu$s of `chrony` agreement to sub-microsecond, but behind a one-millisecond stamp, $70\,\mu$s and 70 ns produce byte-identical records: the guard deletes the same samples and the grid law is unchanged. The remedies are therefore ordered, record at native resolution, count what is discarded, dither the send instant, and only then does synchronisation quality become the frontier. Our cross-host data makes the same point from the other side, clocks agreeing to well under a tick while retention wandered 13.4 to 27.0% with offset drift, so at minimum the synchronisation state belongs in the published record beside the retention rate. Where one-way synchronisation is genuinely required, designs that remove the requirement predate the clouds needing them: round-trip measurement on one clock, offset and skew removal from the delay envelope [1], and software-only sub-microsecond synchronisation where probe traffic is available [15].

One redesign deserves a direct answer, because it is the first thing a careful reader proposes: have the consumer acknowledge back to the producer, so a single clock times a round trip. It removes skew by construction, and it does not remove this failure. The quantity changes, since a producer-side round trip is not the staleness of an event at the consumer, which is what a streaming pipeline is judged on; and the stall does not care about clock count, since the arm where both processes read the same clock is the arm the audit rejects most (62.4%). A same-clock protocol still needs its stamps audited.

C. Threats to validity

A negative span proves something is wrong; the attribution to scheduling is an inference, separated from its rivals by manipulation rather than assumed, and the eliminations are listed at the end of Section V. Clock skew is ruled out by construction on the same-clock arms and bounded on the others; the observed clustering of consecutive late wakes ($z = -6.9$ at the co-located floor, at every load including none) is the signature of a shared scheduling delay rather than independent kernel granularity. `chrony` reports the inter-host offset of Section III-B, comfortably sub-tick, but the bound it places on its own error, root dispersion plus half the root delay, is 4 to 7 ms per host, so a pair may disagree by up to 12 ms against a 1 ms timestamp. That leaves the cross-host channel open rather than ruled out. It is not what produced the Redis-driver negatives, which the driver's own choice of stamp accounts for exactly (Section IV-G), but it is why we claim no bound on cross-host exposure in general.

D. Limitations

The mechanism's constants are those of one EEVDF-era kernel (6.8); CFS-era schedulers, the regime Lozi et al. document, may shift them. The deletion law is demonstrated on one benchmark and reproduced in one independent harness; the pattern it requires, a quantised stamp, a paced producer and a positivity guard, is common, but we audited no deployment. The guard we analyse is not confined to the tool we measured: the same expression appears unchanged in more than a dozen public forks and in tooling released in 2026, so the exposure is current rather than historical. The OpenMessaging distributed mode failed identically on every attempt, so the cross-host exposure of that particular tool rests on source reading plus our own two-host harness rather than on running it. Exposure in any given deployment depends on its path latency and producer rate, which is why we recommend publishing a retention rate as routine practice rather than claiming any published number is wrong.

VIII. Conclusion

A latency benchmark measures the system only while the instrument is faster than the thing being measured. Below that line two failures appear, and both are invisible in the output. A stamp written by a thread that must first be scheduled can arrive after the event it was meant to mark, and the difference inverts: the sign check rejected 1,321 of 2,266 of our own runs, and across 738,730 events the acknowledgement-referenced span inverted 62,264 times while the send-referenced span never did, on one clock, with real-time priority collapsing the rate 7–80$\times$ at fixed load. A stamp quantised to a millisecond and filtered for positivity deletes samples by an arithmetic the operator never sees: 0.36% to 100% of the data behind one unchanged median. Neither failure needs a better clock to fix, and neither is expensive to detect. Check the spans whose endpoints are stamped in different places, count what your instrument throws away, and publish the retention rate beside the number.

Acknowledgment

The author thanks R. Duvignau, whose reading established that the acknowledgement-referenced span is not a causal chain and prompted the recount that reorganised this paper; D. Gregg, for the readability critique that shaped the previous version and for the self-consistency argument in Section IV; J. Kunkel, for the same-clock protocol contrast, the offset-estimation route and the measurement-model figure; N. Herbst, for load-generation validation and the timer-characterisation literature; M. Luthra, for the kernel-tracing pointers; M. Kogias, for the probe-placement argument; A. Ousterhout, for the preemptibility scope condition; and L. Tratt, for the question about timestamp resolution answered in Section IV.

Artefact Availability

Code, analysis and manuscript are archived at 10.5281/zenodo.21836305 and the measurement dataset at 10.5281/zenodo.21836326. Every number in this paper is recomputed from the committed data by the archived code at build time, and the test suite fails if the text and the data disagree.

References

[1] V. Paxson, "On calibrating measurements of packet transit times," in *Proc. ACM SIGMETRICS*. ACM, 1998, pp. 11–21.

[2] P. Dobbelaere and K. S. Esmaili, "Kafka versus RabbitMQ: A comparative study of two industry reference publish/subscribe implementations," in *Proceedings of the 11th ACM International Conference on Distributed and Event-Based Systems (DEBS)*. ACM, 2017, pp. 227–238.

[3] OpenMessaging Project, Linux Foundation, "The OpenMessaging benchmark framework," 2018, available at openmessaging.cloud/docs/benchmarks/; accessed 19 August 2026.

[4] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!" in *Proc. 14th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2009, pp. 265–276.

[5] E. Barrett, C. F. Bolz-Tereick, R. Killick, S. Mount, and L. Tratt, "Virtual machine warmup blows hot and cold," *Proceedings of the ACM on Programming Languages*, vol. 1, no. OOPSLA, pp. 52:1–52:27, 2017.

[6] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Programming, Systems, Languages and Applications (OOPSLA)*. ACM, 2007, pp. 57–76.

[7] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC)*. ACM, 2015, pp. 73:1–73:12.

[8] T. Kalibera and R. Jones, "Rigorous benchmarking in reasonable time," in *Proceedings of the 2013 International Symposium on Memory Management (ISMM)*. ACM, 2013, pp. 63–74.

[9] B. Schroeder, A. Wierman, and M. Harchol-Balter, "Open versus closed: a cautionary tale," in *Proceedings of the 3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. ACM, 2006, pp. 239–252.

[10] J. Karimov, T. Rabl, A. Katsifodimos, R. Samarev, H. Heiskanen, and V. Markl, "Benchmarking distributed stream data processing systems," in *Proc. IEEE 34th Int. Conf. Data Engineering (ICDE)*, 2018, pp. 1507–1518.

[11] G. Van Dongen and D. Van den Poel, "Evaluation of stream processing frameworks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 8, pp. 1845–1858, 2020.

[12] M. Fruth, S. Scherzinger, W. Mauerer, and R. Ramsauer, "Tell-tale tail latencies: Pitfalls and perils in database benchmarking," in *Performance Evaluation and Benchmarking (TPCTC)*. Springer, 2021, pp. 119–134.

[13] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.

[14] IEEE, "IEEE standard for a precision clock synchronization protocol for networked measurement and control systems," IEEE Std 1588-2019, 2020, revision of IEEE Std 1588-2008.

[15] Y. Geng, S. Liu, Z. Yin, A. Naik, B. Prabhakar, M. Rosenblum, and A. Vahdat, "Exploiting a natural network effect for scalable, fine-grained clock synchronization," in *Proc. 15th USENIX NSDI*. USENIX Association, 2018, pp. 81–94.

[16] S. Yang, J. Jeong, B. Scholz, and B. Burgstaller, "Cloudprofiler: TSC-based inter-node profiling and high-throughput data ingestion for cloud streaming workloads," *arXiv preprint arXiv:2205.09325*, 2022.

[17] M. Kogias, S. Mallon, and E. Bugnion, "Lancet: A self-correcting latency measuring tool," in *Proc. USENIX Annual Technical Conf. (ATC)*, 2019, pp. 881–896.

[18] R. Kundel, "P4STA: High precision network function benchmarking," in *Accelerating Network Functions Using Reconfigurable Hardware*, ser. Springer Theses. Springer Nature Switzerland, 2024, pp. 93–115.

[19] M. Kuperberg, M. Krogmann, and R. H. Reussner, "Metric-based selection of timer methods for accurate measurements," in *Proceedings of the 2nd ACM/SPEC International Conference on Performance Engineering (ICPE)*, Mar. 2011, pp. 151–156.

[20] D. Bristot de Oliveira, D. Casini, J. Lelli, and T. Cucinotta, "Timerlat: Real-time Linux scheduling latency measurements, tracing, and analysis," *IEEE Trans. Comput.*, vol. 74, no. 8, pp. 2608–2620, 2025.

[21] B. Villain, M. Davis, J. Ridoux, D. Veitch, and N. Normand, "Probing the latencies of software timestamping," in *Proc. IEEE Int. Symp. Precision Clock Synchronization for Measurement, Control and Communication (ISPCS)*, 2012, pp. 1–6, dTrace breakdown against a DAG hardware reference; reports software stamps that "do not respect causality" under load.

[22] A. Sharma, D. Shah, D. Lariviere, and H. ElBakoury, "Time, causality, and observability failures in distributed AI inference systems," *arXiv preprint arXiv:2604.21361*, Apr. 2026, open Compute Project, Unified Intelligent Infrastructure workstream.

[23] A. Swami and D. Sonawane, "Wire-level interrupt-to-decision latency of on-sensor MLC versus host inference on the NVIDIA Jetson Orin Nano: A pre-registered measurement study," *arXiv preprint arXiv:2606.00524*, May 2026.

[24] P. B. Danzig and S. Melvin, "High resolution timing with low resolution clocks and a microsecond resolution timer for sun workstations," *ACM SIGOPS Operating Systems Review*, vol. 24, no. 1, pp. 23–26, Jan. 1990.

[25] J. C. Gust, R. M. Graham, and M. A. Lombardi, "Stopwatch and timer calibrations (2009 edition)," National Institute of Standards and Technology, NIST Special Publication 960-12, 2009, uncertainty budget carrying both operator reaction time and display resolution.

[26] V. Paxson, G. Almes, J. Mahdavi, and M. Mathis, "Framework for IP performance metrics," Internet Engineering Task Force, RFC 2330, May 1998.

[27] V. Raisanen, G. Grotefeld, and A. Morton, "Network performance measurement with periodic streams," Internet Engineering Task Force, RFC 3432, Nov. 2002.

[28] G. Tene, "How NOT to measure latency," 2015, talk, Strange Loop / QCon; coined "coordinated omission".

[29] M. Wiederhold, "Sub-millisecond latencies are reported as taking 0 milliseconds (issue #41)," Yahoo! Cloud Serving Benchmark, Oct. 2011, https://github.com/brianfrankcooper/YCSB/issues/41; fixed by pull request #42 and released in 0.1.4.

[30] L. Schuchman, "Dither signals and their effect on quantization noise," *IEEE Transactions on Communication Technology*, vol. 12, no. 4, pp. 162–165, 1964.

[31] J.-P. Lozi, B. Lepers, J. Funston, F. Gaud, V. Quéma, and A. Fedorova, "The Linux scheduler: a decade of wasted cores," in *Proc. 11th European Conf. Computer Systems (EuroSys)*. ACM, 2016, pp. 1–16.

[32] A. Chandrasekar and J. Kramberger, "Identifying and mitigating systemic measurement bias in production LLM inference benchmarks," *arXiv preprint arXiv:2605.24217*, 2026.

[33] J. Li, N. K. Sharma, D. R. K. Ports, and S. D. Gribble, "Tales of the tail: Hardware, OS, and application-level sources of tail latency," in *Proc. ACM SoCC*. ACM, 2014.

[34] M. Kuperberg, M. Krogmann, and R. H. Reussner, "TimerMeter: Quantifying properties of software timers for system analysis," in *Proc. 6th Int. Conf. Quantitative Evaluation of Systems (QEST)*, 2009, pp. 85–94.

[35] J. L. Hodges and E. L. Lehmann, "Estimates of location based on rank tests," *The Annals of Mathematical Statistics*, vol. 34, no. 2, pp. 598–611, 1963.

[36] D. J. Schuirmann, "A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability," *Journal of Pharmacokinetics and Biopharmaceutics*, vol. 15, no. 6, pp. 657–680, 1987.

[37] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[38] Y. Virkar and A. Clauset, "Power-law distributions in binned empirical data," *The Annals of Applied Statistics*, vol. 8, no. 1, pp. 89–119, 2014.

[39] Apache Kafka, "KIP-32: Add timestamps to Kafka message," Apache Software Foundation, Kafka Improvement Proposal, 2015, defines the default producer-set CreateTime.

[40] S. Guo, "Avoid adding negative metric values (pull request #56)," OpenMessaging Benchmark, commit ebb5d6b8, Mar. 2018, https://github.com/openmessaging/benchmark/pull/56; accessed 18 August 2026.

[41] Apache Kafka, "KIP-489: Kafka consumer record latency metric," Apache Software Foundation, Kafka Improvement Proposal, 2019.

[42] T. Downs, "Grow buffer if histogram can't fit (pull request #398)," OpenMessaging Benchmark, commit 4eb2327c, Dec. 2023, fixes issue #369: truncated histogram serialisation silently discarded most latency samples. https://github.com/openmessaging/benchmark/pull/398; accessed 19 August 2026.

[43] B. Gregg, *BPF Performance Tools: Linux System and Application Observability*. Addison-Wesley, 2019, `runqlat`, the run-queue latency histogram used here.

[44] S. Shalunov, B. Teitelbaum, A. Karp, J. W. Boote, and M. J. Zekauskas, "A one-way active measurement protocol (OWAMP)," Internet Engineering Task Force, RFC 4656, Sep. 2006.

[45] G. Almes, S. Kalidindi, M. Zekauskas, and A. Morton, "A one-way delay metric for IPPM," Internet Engineering Task Force, RFC 7679, Jan. 2016, obsoletes RFC 2679. Section 3.5 keeps negative values as distribution data rather than discarding them.

Gustavo Pedro Ricou received the B.Sc. degree in physics from the Universidade do Porto, Portugal, and specialised in nuclear fusion science at the University of Stuttgart, Germany, and the Université de Lorraine, France. He received the Higher Diploma in computing from Atlantic Technological University and an M.Sc. in software design with cloud-native computing from the Technological University of the Shannon, Ireland, both with first-class honours, and is currently pursuing an M.Sc. in statistics and data science at Trinity College Dublin, Ireland.

He is a Software Engineer with BoscaSports, building real-time pipelines for live sports, with prior roles at Invertase, Xerotech, Datto and TikTok. His research concerns measurement validity — what a reported number is evidence of, and when a routine convention distorts what it summarises — in computer systems, environmental measurement and sports analytics, using pre-registration and physical-consistency auditing.